

Unit 1: Introduction to software engineering [5hr]

1.1 Overview of software engineering and its application domain

Software:

- Software is a program or set of programs containing instructions that provide the desired functionality.

Engineering:

- It is an application of science, tools and methods to find the cost-effective solution to the problem.

Software engineering:

- **It is the process of designing, developing, testing, and maintaining software.** Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance. Software engineering applies systematic, disciplined, and quantifiable engineering principles to the design, development, testing, and maintenance of software systems to create high-quality, reliable products.

The IEEE(Institute of Electrical and Electronics Engineers) definition:

The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

Why do we need software engineering?

- Software systems are becoming **larger, more complex**, and often critical (for business, safety, mission-critical).
- Without a disciplined approach, projects tend to go over budget, miss deadlines, have many defects, and maintenance becomes impossible.
- Software engineering provides frameworks, methods, and tools to **manage complexity, improve quality, reduce risks**, and deliver reliable software on time and budget.

Software Application Domain:

- Software application domains are the specific subject areas or fields where a software system is developed and used, such as web development, mobile app development, game development, or embedded systems.
- A software domain defines the specific field or environment where a software product operates or provides solutions.
- It classifies software based on its purpose, characteristics, and the environment in which it operates.

1. **System software:** It is a collection of programs written to service other programs. Software designed to manage and control computer hardware and provide a platform for running application programs. For example: Operating Systems (Windows, Linux), Compilers, Device Drivers, built-in editors, file management utilities
2. **Application software:** Programs developed to solve specific business or real-world problems for users. stand-alone programs for specific needs. For example: MS Word, Payroll System, Inventory System
3. **Engineering/scientific software:** Characterized by “number crunching” algorithms. Used for scientific research, engineering analysis, and computational tasks requiring high precision. For example: MATLAB, AutoCAD, Simulation Software used for automotive stress analysis, molecular biology, orbital dynamics etc.
4. **Embedded software:** Software built into hardware devices to control their functions. Resides within a product or system. (washing machine, key pad control of a microwave oven, digital function of dashboard display in a car)
5. **Product-line software:** Designed to meet the needs of a specific market segment or customer group. Focus on a limited marketplace to address mass consumer market. (word processing, graphics, database management).
6. **Web / mobile application:** Software designed for online environments (internet/intranet), including mobile and cloud-based systems.
 - Web app is the applications that run over the internet using a browser.
 - Often use client-server architecture.
 - Include social media, e-commerce, and online services.
 - Software designed to run on mobile devices (Android, iOS).
 - Focus on user experience, connectivity, and mobility.
7. **AI software:** Software that exhibits intelligent behavior like learning, reasoning, and decision-making. Uses non-numerical algorithm to solve complex problem. For example: Chatbots, Robotics, expert system, pattern recognition, Recommendation Systems.

Application of Software Engineering:

1. Web and Mobile Application Development

- Modern software engineering practices are widely used to create web and mobile apps that are user-friendly, responsive, and secure. This includes e-commerce platforms, social

media apps, and productivity tools that need robust backend systems, intuitive user interfaces, and reliable performance.

2. Banking and Finance

- Financial systems, such as online banking, stock trading platforms, and payment processing, heavily rely on software engineering for security, accuracy, and speed. These applications handle sensitive data and financial transactions, requiring strict adherence to security and compliance standards.

3. Healthcare and Medical Systems

- Healthcare software, such as Electronic Health Records (EHR), diagnostic systems, and telemedicine platforms, benefits from software engineering principles to ensure data security, patient privacy, and high system availability. Reliability and accuracy are crucial for supporting medical professionals and enhancing patient care.

4. Artificial Intelligence and Machine Learning

- Software engineering plays a key role in developing AI and machine learning applications, such as natural language processing, image recognition, and predictive analytics. Engineering principles are used to create scalable, efficient algorithms and ensure that AI systems are robust and interpretable.

5. Gaming and Multimedia

- The gaming industry relies on software engineering for graphics rendering, gameplay mechanics, and performance optimization. Engineering practices help ensure games are visually appealing, stable, and can handle complex computations in real time.

6. Education and E-Learning

- E-learning platforms, virtual classrooms, and digital content libraries use software engineering for accessibility, scalability, and an engaging user experience. Applications like learning management systems (LMS) enable students and educators to connect, learn, and share resources seamlessly.

1.2 Software process models

❖ Software process

When you work to build a product and system, it's important to go through a series of predictable steps – a road map that you follow is called a “software process”.

A software process is a framework which is collection of **activities**, **actions**, and **tasks** that are performed when some high-quality work product is to be created. In software engineering, software process is adaptable approach that enables the software team (people doing the work) to pick and choose the appropriate set of work actions and tasks. The intent is always to deliver software in a timely manner and with sufficient quality to satisfy those who have sponsored its creation and those who will use it.

An **activity** strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.

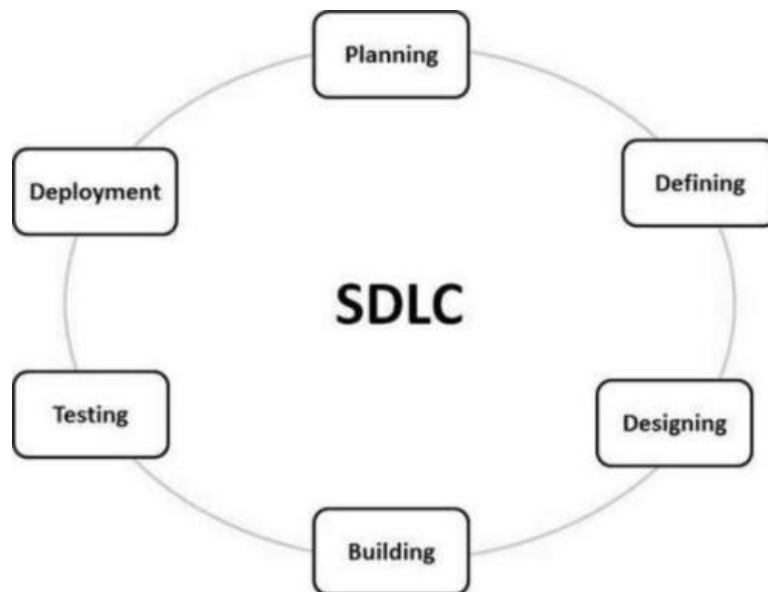
An **action** (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural model).

A **task** focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

A software process is needed because it provides stability, control, and organization to an activity that was left uncontrolled, became quite chaotic.

Software Development Life Cycle (SDLC)

- SDLC is a process followed for a software project, within a software organization. It consists of a detailed plan describing how to develop, maintain, replace and alter or enhance specific software. The life cycle defines a methodology for improving the quality of software and the overall development process.
- The following figure is a graphical representation of the various stages of a typical SDLC.



A typical Software Development Life Cycle consists of the following stages –

Stage 1: Planning and Requirement Analysis

- Requirement analysis is the most important and fundamental stage in SDLC. It is performed by the senior members of the team with inputs from the customer, the sales department, market surveys and domain experts in the industry. This information is then used to plan the basic project approach and to conduct product feasibility study in the economical, operational and technical areas.
- Planning for the quality assurance requirements and identification of the risks associated with the project is also done in the planning stage. The outcome of the technical
- feasibility study is to define the various technical approaches that can be followed to implement the project successfully with minimum risks.

Stage 2: Defining Requirements

- Once the requirement analysis is done the next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts. This is done through an **SRS (Software Requirement Specification)** document which consists of all the product requirements to be designed and developed during the project life cycle.

Stage 3: Designing the Product Architecture

- SRS is the reference for product architects to come out with the best architecture for the product to be developed. Based on the requirements specified in SRS, usually more than one design approach for the product architecture is proposed and documented in a **DDS - Design Document Specification**.
- This DDS is reviewed by all the important stakeholders and based on various parameters as risk assessment, product robustness, design modularity, budget and time constraints, the

best design approach is selected for the product.

Stage 4: Building or Developing the Product

- In this stage of SDLC the actual development starts and the product is built. The programming code is generated as per DDS during this stage. If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.
- Developers must follow the coding guidelines defined by their organization and programming tools like compilers, interpreters, debuggers, etc. are used to generate the code. Different high level programming languages such as C, C++, Pascal, Java and PHP are used for coding. The programming language is chosen with respect to the type of software being developed.

Stage 5: Testing the Product

- This stage refers to the testing only stage of the product where product defects are reported, tracked, fixed and retested, until the product reaches the quality standards defined in the SRS.

Stage 6: Deployment in the Market and Maintenance

- Once the product is tested and ready to be deployed it is released formally in the appropriate market. Sometimes product deployment happens in stages as per the business strategy of that organization. The product may first be released in a limited segment and tested in the real business environment (**UAT- User acceptance testing**).
- Then based on the feedback, the product may be released as it is or with suggested enhancements in the targeting market segment. After the product is released in the market, its maintenance is done for the existing customer base.

1.2 SOFTWARE PROCESS MODELS

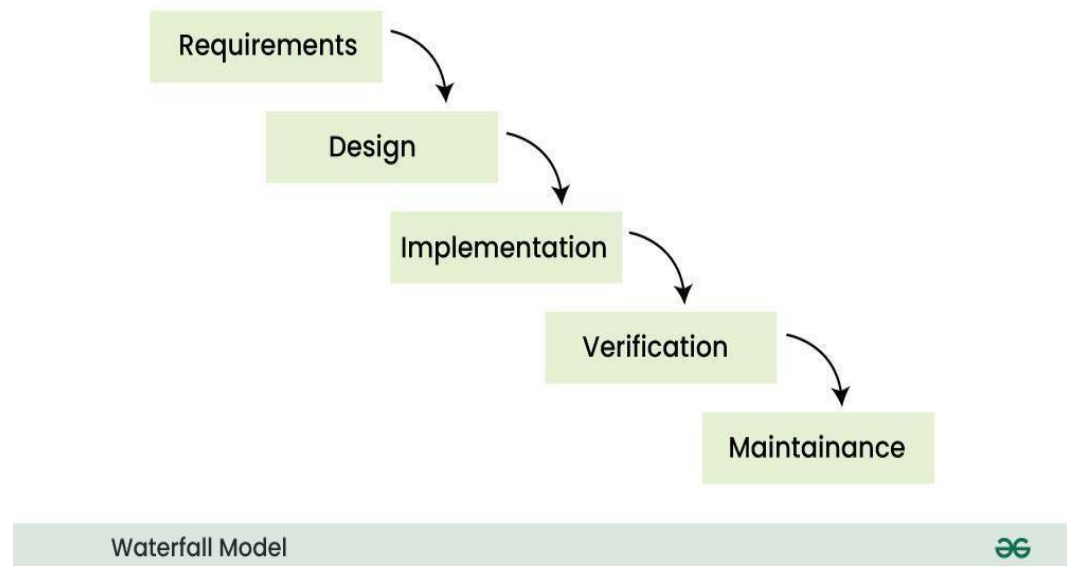
A Process model is something that provides a specific roadmap for software engineering work. It defines the flow all activities, actions and tasks, the degree of iteration, the work products, and the organization of the work that must be done.

There are different types of software process models, some of them are explain below:

I. Waterfall models

The Waterfall model is a linear and sequential approach to software development where each phase must be completed before the next one begins. This model follows a strict order: starting with Requirement Gathering, then moving to Design, Implementation, Testing, Deployment, and finally, Maintenance. Once a phase is completed, it typically cannot be revisited, making the process rigid and structured.

The Waterfall model is best suited for projects with well-defined requirements and minimal expected changes, as it emphasizes careful planning and a clear progression through each phase. While this model provides strong documentation and a predictable workflow.



Requirement phase: - Requirement phase is the first phase of the waterfall model. In this phase the requirements of the system are collected and documented. This phase is very crucial because the next phases are based on this phase.

Design/ Planning phase: - Design phase is based on the fact how the software will be built. The main objective of the design phase is to prepare the blueprint of the software system so that no problems are faced in the coming phases and solutions to all the requirements in the requirement phase are found.

Implementation phase: - In this phase, hardware, software and application programs are installed and the database design is implemented. Before the database design can be implemented, the software has to go through a testing, coding, and debugging process. This is the longest lasting phase in waterfall.

Verification/ Testing phase: - In this phase the software is verified and it is evaluated that we have created the right product. In this phase, various types of testing are done and every area of the software is checked. It is believed that if we do not verify the software properly and there is any defect in it then no one will use it, hence verification is very important. One advantage of verification is that it reduces the risk of software failure.

Deployment and Maintenance phase: - This is the last phase of waterfall. When the system is ready and users start using it, then the problems that arise have to be solved time-to-time. Taking care of the finished software and maintaining it as per time is called maintenance.

Real time example: Payroll system, result processing system

Advantages of Waterfall Model

- This model is simple and easy to understand.
- This is very useful for small projects.
- This model is easy to manage.
- The end goal is determined early.
- Each phase of this model is well explained.

Disadvantages of Waterfall Model

- In this model, complete and accurate requirements are expected at the beginning of the development process.
- Working software is not available for very long during the development life cycle.
- We cannot go back to the previous phase due to which it is very difficult to change the requirements.
- High Risk

When to use SDLC Waterfall Model?

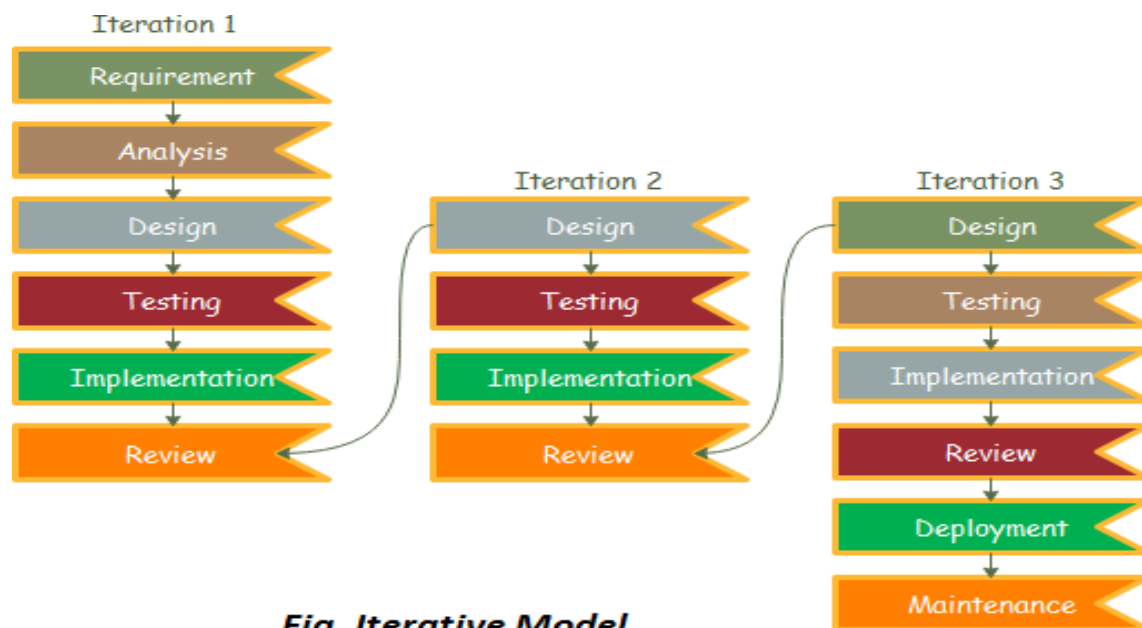
Some Circumstances where the use of the Waterfall model is most suited are:

- When the requirements are constant and not changed regularly.
- A project is short
- The situation is calm
- Where the tools and technology used is consistent and is not changing
- When resources are well prepared and are available to use.

II. Iterative Model

In this Model, you can start with some of the software specifications and develop the first version of the software. After the first version if there is a need to change the software, then a new version of the software is created with a new iteration. Every release of the Iterative Model finishes in an exact and fixed period that is called iteration.

The Iterative Model allows the accessing earlier phases, in which the variations made respectively. The final output of the project renewed at the end of the Software Development Life Cycle (SDLC) process.



1. Requirement gathering & analysis: In this phase, requirements are gathered from customers and check by an analyst whether requirements will fulfil or not. Analyst checks that need will achieve within budget or not. After all of this, the software team skips to the next phase.

2. Design: In the design phase, team design the software by the different diagrams like Data Flow diagram, activity diagram, class diagram, state transition diagram, etc.

3. Implementation: In the implementation, requirements are written in the coding language and transformed into computer programs which are called Software.

4. Testing: After completing the coding phase, software testing starts using different test methods. There are many test methods, but the most common are white box, black box, and grey box test methods.

5. Deployment: After completing all the phases, software is deployed to its work environment.

6. Review: In this phase, after the product deployment, review phase is performed to check the behavior and validity of the developed product. And if there are any error found then the process starts again from the requirement gathering.

7. Maintenance: In the maintenance phase, after deployment of the software in the working environment there may be some bugs, some errors or new updates are required. Maintenance involves debugging and new addition options.

Real time example: E-commerce website, Simple game development.

Advantages of Iterative Model:

- Testing and debugging during smaller iterations are easy.
- A Parallel development can plan.
- Risks are identified and resolved during iteration.
- Limited time spent on documentation and extra time on designing.

Disadvantages of Iterative Model:

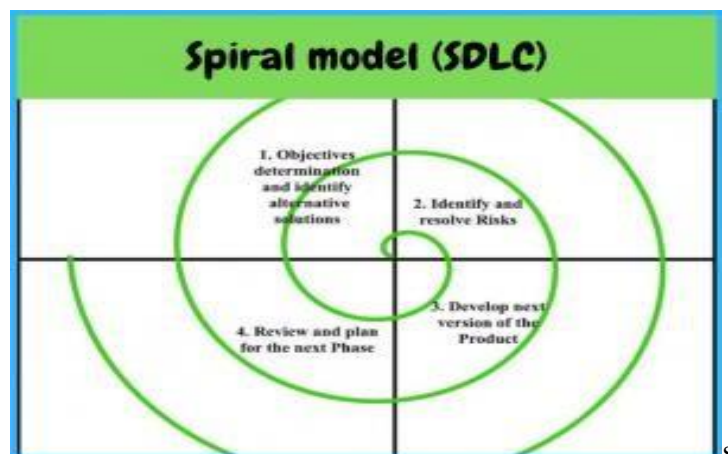
- It is not suitable for smaller projects.
- More Resources may be required.
- Requirement changes can cause over budget.
- Project completion date not confirmed because of changing requirements.

When to use the Iterative Model?

1. When requirements are defined clearly and easy to understand.
2. When the software application is large.
3. When there is a requirement of changes in future.

III. Spiral Model

The spiral model is a software development process model was originally proposed by Barry Boehm. This model has characteristics of both iterative and waterfall models. This model is used in projects which are large and complex. This model was named spiral because if we look at its figure, it looks like a spiral, in which a long-curved line starts from the center point and makes many loops around it. The number of loops in the spiral is not decided in advance but it depends on the size of the project and the changing requirements of the user. We also call each loop of the spiral a phase of the software development process.



- **Determining objectives and alternate solutions:** In the first phase, whatever requirements the customer has related to the software are collected. On the basis of which objectives are identified and analyzed and various alternative solutions are proposed.
- **Identifying and resolving risks:** In this phase, all the proposed solutions are assessed and the best solution is selected. Now that solution is analyzed and the risks related to it are identified. Now the identified risks are resolved through some best strategy.
- **Develop and test:** Now the development of software is started. In this phase various features are implemented, that is, their coding is done. Then those features are verified through testing.
- **Review and plan for the next phase:** In this phase the developed version of the software is given to the customer and he evaluates it. Gives his feedback and tells new

requirements. Finally planning for the next phase (next spiral) is started.

Real time example: Flight control, banking, or defense system

When to use Spiral Model?

- When deliverance is required to be frequent.
- When the project is large
- When requirements are unclear and complex
- When changes may require at any time
- Large and high budget projects

Advantages

- High amount of risk analysis
- Useful for large and mission-critical projects.

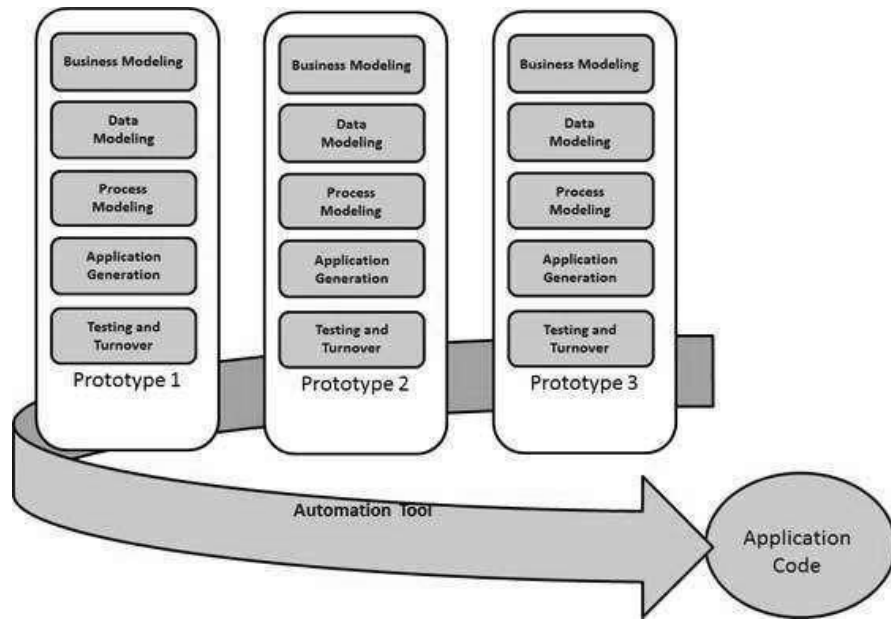
Disadvantages

- Can be a costly model to use.
- Risk analysis needed highly particular expertise
- Doesn't work well for smaller projects.

IV. Rapid Application Development

Rapid Application Development (RAD) is an incremental software development process model that emphasizes an extremely short development cycle. The RAD model is a “high-speed” adaptation of the linear sequential model in which rapid development is achieved by using component-based construction.

- RAD process enables a development team to create a “fully functional system” within very short time periods (e.g., 60 to 90 days).
- In RAD model, the components or functions are developed in parallel as if they were mini projects.
- This can quickly give the customer something to see and use and to provide feedback regarding the delivery and their requirements.
- The main goal of RAD Model is to make the reusability of the developed components.



Different phases of RAD model includes

Phases of RAD model	Activities performed in RAD Model
Business Modeling	On basis of the flow of information and distribution between various business channels, the product is designed
Data Modeling	The information collected from business modeling is refined into a set of data objects that are significant for the business
Process Modeling	The data object that is declared in the data modeling phase is transformed to achieve the information flow necessary to implement a business function
Application Generation	Automated tools are used for the construction of the software, to convert process and data models into prototypes
Testing and Turnover	As prototypes are individually tested during every iteration, the overall testing time is reduced in RAD.

Real time example: Small business website or mobile app prototype.

Advantages of the RAD model:

- Reduced development time.
- Quick initial reviews occur.
- Encourages customer feedback.

- Integration from very beginning solves a lot of integration issues.

Disadvantages:

- Depends on strong team and individual performances for identifying business requirements.
- Only system that can be modularized can be built using RAD.
- Requires highly skilled developers/designers.
- High dependency on modeling skills.
- Inapplicable to cheaper projects as cost of modeling and automated code generation is very high.

When to use RAD model:

- RAD should be used when there is a need to create a system that can be modularized in 2-3 months of time.
- When technical risk is less.
- When the requirements are known.
- It should be used if there's high availability of designers for modeling and the budget is high enough to afford their cost along with the cost of automated code generating tools.
- RAD SDLC model should be chosen only if resources with high business knowledge are available and there is a need to produce the system in a short span of time (2-3 months).

V. Reuse-Oriented Software Engineering (ROSE) model is a development approach that focuses on building software systems by reusing existing software components rather than creating everything from scratch. This model aims to save time, reduce costs, and improve reliability by incorporating pre-built modules, libraries, frameworks, or even entire software packages where possible.

The ROSE model typically involves the following steps:

Component Analysis: Identifying and evaluating existing components to see if they meet the project's requirements.

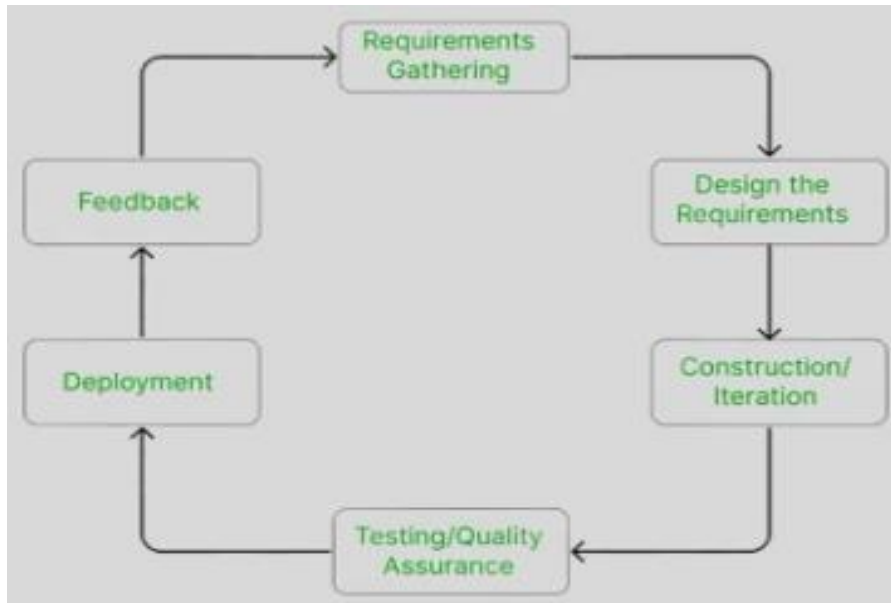
Requirement Modification: Adjusting requirements slightly, if necessary, to better align with available components.

System Design with Reuse: Designing the software architecture to integrate the chosen components effectively.

Development and Integration: Writing new code where needed and integrating the reused components into a cohesive system.

1.3 Agile Software development Model:

Agile software development is a methodology that emphasizes flexibility, collaboration, and iterative progress in building software. Unlike traditional approaches that follow a strict sequence of phases (like Waterfall), Agile is more adaptable and allows teams to deliver smaller, functional parts of the software incrementally.



Steps in Agile Model

1. **Requirement Gathering:** - In this step, the development team must gather the requirements, by interaction with the customer. development team should plan the time and effort needed to build the project. Based on this information you can evaluate technical and economic feasibility.
2. **Design the Requirements:** - In this step, the development team will use user-flow-diagram or high-level UML diagrams to show the working of the new features and show how they will apply to the existing software. Wireframing and designing user interfaces are done in this phase.
3. **Construction / Iteration:** - In this step, development team members start working on their project, which aims to deploy a working product.
4. **Testing / Quality Assurance:** - Testing involves Unit Testing, Integration Testing, and System Testing. A brief introduction of these three tests is as follows:
 - **Unit Testing:** - Unit testing is the process of checking small pieces of code to ensure that the individual parts of a program work properly on their own. Unit

testing is used to test individual blocks (units) of code.

- **Integration Testing:** - Integration testing is used to identify and resolve any issues that may arise when different units of the software are combined.
 - **System Testing:** - Goal is to ensure that the software meets the requirements of the users and that it works correctly in all possible scenarios.
5. **Deployment:** - In this step, the development team will deploy the working project to end users.
 6. **Feedback:** - This is the last step of the **Agile Model**. In this, the team receives feedback about the product and works on correcting bugs based on feedback provided by the customer.

Agility Principles

The Agile Alliance defines 12 agility principles for those who want to achieve agility:

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Agile Frameworks:

❖ Scrum

Scrum is an agile framework for managing complex projects through an iterative and incremental approach. It uses a defined set of roles, events, and artifacts to help teams work collaboratively in short, time-boxed cycles called sprints. The goal is to deliver value frequently, allowing for rapid adaptation based on feedback. Once a sprint starts, its timeline and goals cannot be changed. But once the sprint end, changes are welcome.

Key components:

Roles: A Scrum team includes a Product Owner, a Scrum Master, and the Development Team.

Events: The work happens within a time-boxed period called a sprint, which includes events like the Sprint Planning, the Daily Scrum, the Sprint Review, and the Sprint Retrospective (A meeting to improve process after each sprint.).

Artifacts: Artifacts are essential pieces of information or documentation that represent work or value. These provide transparency and focus on the work. Key artifacts include the Product Backlog (a list of all features), the Sprint Backlog (tasks for the current sprint), and the Increment (the usable or working product).

❖ **Extreme Programming (XP)**

XP is one of the most well-known and disciplined agile software development methodologies. It is focused on improving software quality and enhancing the responsiveness to changing customer requirement. XP doesn't rely on long-term, fixed plans. Instead, it follows a continuous cycle driven by customer value and immediate feedback. It emphasizes practical engineering practices that can be directly applied to software development and the team has to follow a strict priority order or pre-determined priority order.

XP Values

Beck defines a set of five values that establish a foundation for all work performed as part of XP—communication, simplicity, feedback, courage, and respect. Each of these values is used as a driver for specific XP activities, actions, and tasks.

Communication: In the XP context, a metaphor is “a story that everyone—customers, programmers, and managers— can tell about how the system works”. The important concepts and feedback from metaphor help to establish required features and functions for the software.

Simplicity: To achieve simplicity, XP restricts developers to design only for immediate needs, rather than consider future needs. The intent is to create a simple design that can be easily implemented in code. If the design must be improved, it can be refactored at a later time. Refactoring allows a software engineer to improve the internal structure of a design (or source code) without changing its external functionality or behavior.

Feedback: Feedback is derived from three sources: the implemented software itself, the customer, and other software team members. By designing and implementing an effective testing strategy, the software (via test results) provides the agile team with feedback.

Courage or Discipline: There is often significant pressure to design for future requirements because most software teams arguing that “designing for tomorrow” will save time and effort in the long run. But, an agile XP team must have the discipline (courage) to design for today, recognizing that future requirements may change dramatically, thereby demanding substantial rework of the design and implemented code.

Respect: By following each of these four values, the agile team inculcates(teach) respect

among its members, stakeholders and team members, and indirectly, for the software itself. As they achieve successful delivery of software increments, the team develops growing respect for the XP process.

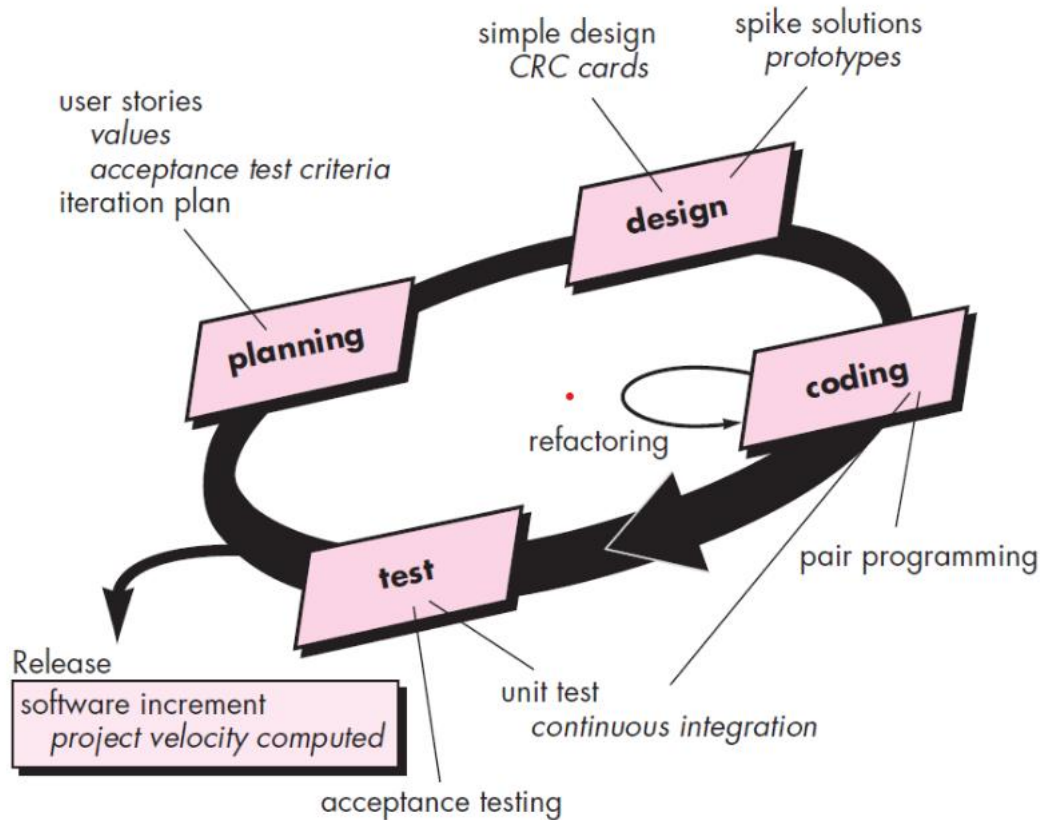


Fig. The Extreme Programming process

1.4 Software Crisis and Myths

Software Crisis:

- The term “software crisis” refers to the numerous challenges and difficulties faced by the software industry during the 1960s and 1970s. It became clear that old methods of developing software couldn’t keep up with the growing complexity and demands of new projects. This led to high costs, delays, and poor-quality software. New methodologies and tools were needed to address these issues.
- **Software Crisis** is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time. The software crisis was due to using the

same workforce, same methods, and same tools even though rapidly increasing software demand, the complexity of software, and software challenges. With the increase in software complexity, many software problems happened because existing methods were insufficient.

Causes of Software Crisis:

- Project runs overbudget
- Project runs over time
- Unreliable Software
- Lack of Communication
- Lack of Resource management
- Difficulty in maintaining code

SOFTWARE DEVELOPMENT MYTHS:

Erroneous beliefs about software and the process that is used to built are known as myths about software development.

There are myths at different level who are involved directly and indirectly in software development process:

- **Management myths**
- **Customer myths**
- **Practitioner's (developers) myths**

1. **Management myths:** Managers with software responsibility, like managers who are under pressure to maintain budget, keep schedules from slipping, improve quality. Software mangers often grasp at belief in a software myth, if that belief lessen the pressure.

Myth: We already have a book that is full of standards and procedure for building software. Won't that provide my people with everything they need to know.

Reality: The book of standards may very well exist, but is it used? Are software practitioners aware of its existence? Does it reflect modern software engineering

practice? Is it complete? Is it adaptable? Is it streamlined to improve time-to-delivery while still maintaining a focus on quality? In many cases, the answer to all of these questions are no.

Myth: If we get behind schedule, we can add more programmers and catch up.

Reality: Adding people to a late software project makes it later. As new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort. People can be added but only in a planned and well-coordinated manner.

Myth: If I decide to outsource the software project to a third party, I can just relax and let them firm built it.

Reality: If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.

2. **Customer myths:** In many cases, the customers believe myths about software. Myths lead to false expectation and, ultimately, dissatisfaction with the developer.

Myth: A general statement of objectives is sufficient to begin writing programs – we can fill in the details later.

Reality: Although a comprehensive and stable statement of requirements is not always possible, an ambiguous “statement of objectives” is a recipe for disaster. Unambiguous requirements are developed only through effective and continuous communication between customer and developer.

Myth: Software requirements continually change, but change can be easily accommodated because software is flexible.

Reality: It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When requirements changes are requested early (before design and code has been started), the cost of impact is relatively small. However, as time passes, the cost impact grows rapidly – resources have been committed, a design framework has been established, and change can cause trouble that requires additional resources and major design modification.

3. **Practitioner’s myths:** Myths that are still believed by software practitioners have been fostered by over 60 years of programming culture. Old ways and attitudes die hard.

Myth: Once we write the program and get it to work, our job is done.

Reality: Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.

Myth: Until I get the program “running” I have no way of accessing its quality.

Reality: One of the most effective software quality assurance mechanisms can be applied from the inception of project – the technical review. Software reviews are a “quality filter” that have been found to be more effective than testing for finding certain classes of software defects.

Myth: The only deliverable work product for a successful project is the working product.

Reality: A working program is only one part of software configuration that includes many elements. A variety of work products (e.g., models, documents, plans) provide a foundation for a successful engineering and, more important, guidance for software support.

Myth: Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.

Reality: Software engineering is not about creating documents. It is about creating a quality product. Better quality leads to reduced rework. And reduced rework results in faster deliver.

Four P’s of Software:

People

The most important component of a product and its successful implementation is human resources. In building a proper product, a well-managed team with clear-cut roles defined for each person/team will lead to the success of the product. We need to have a good team in order to save our time, cost, and effort. Some assigned roles in software project planning are project manager, team leaders, stakeholders, analysts, and other IT professionals. Managing people successfully is a tricky process which a good project manager can do.

Process

In every planning, a clearly defined process is the key to the success of any product. It regulates how the team will go about its development in the respective time period. The Process has several steps involved like, documentation phase, implementation phase, deployment phase, and interaction phase.

Project

The last and final P in software project planning is Project. It can also be considered as a blueprint of process. In this phase, the project manager plays a critical role. They are responsible to guide the team members to achieve the project's target and objectives, helping & assisting them with issues, checking on cost and budget, and making sure that the project stays on track with the given deadlines.

Product

As the name inferred, this is the deliverable or the result of the project. The project manager should clearly define the product scope to ensure a successful result, control the team members, as well technical hurdles that he or she may encounter during the building of a product. The product can consist of both tangible or intangible such as shifting the company to a new place or getting a new software in a company.

Assignments

- I. Define software engineering. Explain its software application domains.
- II. Difference between Agile, Waterfall and iterative model.
- III. Explain Spiral model & RAD and why it is used.
- IV. What is Agile? Difference between Scrum and XP.
- V. Define Software Crisis. Explain types of Software Myths.
- VI. Explain the Four P's of software and Importance of each P in software development.
- VII. What are the different phases in SDLC? explain.
- VIII. What are the five values of XP in agile and explain xp in detail with necessary diagram (2023 fall)
- IX. Why do you prefer agile process of software development than conventional method of software development? Explain scrum process of software development with necessary diagram. (2024 spring)

Class discussion

The exam control office of Pokhara university aim to digitize its services, making all required task accessible online. The standard operating procedures for each task are already well-defined. While the project has a reasonable timeline for completion, it must be executed within a limited budget.

Which software process model would be more suitable for developing this software? Analyze and discuss the advantage of the chosen process model.